

Introdução à Ciência da Computação com Python II

Dados estruturados e objetos

REALIZAR APÓS Módulo 3 / Semana 3

Módulos 1 a 3 | 120 minutos | 100 pontos

Curso principal: Introdução à Ciência da Computação com Python Parte 2 |
<https://www.coursera.org/learn/ciencia-computacao-python-conceitos-2>

Conteúdo verificável: Matrizes; strings; modularização; OO; código testável

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Nota de fonte: Módulo 7 é explicitamente extra (Scrapy e PyGame); excluído das provas obrigatórias.

Matrizes e alias

Uma matriz 3×2 foi criada com $m = \begin{bmatrix} 0 & 0 \end{bmatrix} \times 3$. (a) Explique o efeito de $m[0][0] = 7$. (b) Corrija a construção. (c) Implemente soma de matrizes de mesma dimensão sem alterar as entradas.

Desenvolvimento / justificativa / esquema:

Strings e módulos

- (a) Implemente uma função que compara duas strings ignorando maiúsculas e espaços nas pontas. (b) Coloque-a num módulo e mostre uma chamada por import. (c) Diferencie igualdade textual de equivalência semântica entre frases.

Desenvolvimento / justificativa / esquema:

Objeto testável

Crie classe Contador, com valor inicial 0 e método incrementar(n) que não imprime. (a) Mostre dois objetos independentes. (b) Teste incremento. (c) Explique por que armazenar valor como atributo de instância importa.

Desenvolvimento / justificativa / esquema:

Multiplicação matricial

Implemente `produto(A, B)` para matrizes retangulares não vazias compatíveis. (a) Justifique dimensões e a soma linha-coluna. (b) Teste $A = \begin{bmatrix} 1 & 2 \end{bmatrix}$, $B =$

$$\begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

. (c) Retorne `None` para dimensões incompatíveis antes do cálculo.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Matrizes e alias (15 pontos)

As três linhas referenciam a mesma lista, logo a primeira coluna de todas vira 7. Construir linhas independentes, por exemplo `m=[[0, 0] for _ in range(3)]`. Soma:

```
def soma(a, b):  
    return [[a[i][j] + b[i][j] for j in range(len(a[0]))]  
            for i in range(len(a))] if a else []
```

Contrato: matrizes retangulares com iguais dimensões; validar se o contrato não for garantido. Testar vazia, 1×1 e duas linhas e confirmar ausência de compartilhamento acidental.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Strings e módulos (25 pontos)

```
# arquivo comparacao.py  
def iguais(a, b):  
    return a.strip().lower() == b.strip().lower()  
# arquivo principal.py  
import comparacao  
assert comparacao.iguais(' Ola ', 'ola')
```

A normalização especificada não remove acentos, pontuação ou espaços internos; não prova que frases tenham o mesmo significado. Testar diferenças reais e entradas vazias. A modularização evita duplicar a função e permite testes independentes.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Objeto testável (25 pontos)

```
class Contador:  
    def __init__(self):  
        self.valor = 0  
    def incrementar(self, n):  
        self.valor += n  
        return self.valor  
a = Contador(); b = Contador()  
assert a.incrementar(3) == 3  
assert b.valor == 0
```

self.valor pertence a cada instância; um estado mutável compartilhado na classe poderia misturar contadores. Retornar valor torna o método verificável sem capturar saída. Documentar se incrementos negativos são aceitos; o enunciado não os proibiu.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Multiplicação matricial (35 pontos)

```
def produto(a, b):
    if len(a[0]) != len(b):
        return None # contrato: dimensoes incompativeis
    return [[sum(a[i][k]*b[k][j] for k in range(len(b)))
             for j in range(len(b[0]))] for i in range(len(a))]
```

Resultado[[11]]. Se A é $m \times n$ e B é $n \times p$, resultado $m \times p$. A soma percorre a dimensão compartilhada n. Pode substituir comprehensions e sum por três laços; essa alternativa é igualmente válida. Retangularidade é pré-condição explicitada.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.